The Regression Cup

STAT 464/564 team project, Fall 2026: the problem, the rules, and how you are scored

AUTHOR
D. Taylor-Rodriguez

Cup app link: https://dtrstats.shinyapps.io/regression-cup/

Public leaderboard link: https://dtrstats.github.io/regression-cup/

> **The Cup in five lines**
>
> - **Task:** for each of 941 counties, predict a (synthetic) EV adoption rate and give a 90% prediction interval.
> - **Submit:** one CSV with a prediction and an interval per county, on the rate scale or the log scale (your choice), uploaded through the Cup app.
> - **Scored on:** point skill (how close your predictions are) and interval skill (how honest and tight your intervals are), both on the log scale, both relative to a simple benchmark.
> - **Limits:** at most **2 scored submissions per week** per team; checking a file is free and unlimited.
> - **Graded on:** reaching stated thresholds and on your memos, **not** on your rank. The rank is for glory (and a trophy).

1 The problem

A fictional nonprofit pays for public fast chargers in U.S. counties. Its rule of thumb is simple: put chargers where electric vehicles already are, and size the order to the number of EVs on the road.

The nonprofit bought vehicle-registration counts for 2,197 counties. It cannot afford the data for the other 941 counties, but it does know a lot about every county: income, education, commuting, housing, health, and more. It has hired your team to answer one question for each county it could not survey:

> **How many EVs per 1,000 registered vehicles does this county have, and how sure are you?**

Both halves of that question matter. A point prediction alone is not enough: if the nonprofit sizes an order to your prediction and the truth is three times larger, chargers sit overloaded; if it is three times smaller, money sits idle. So for every county you also give a **90% prediction interval**, a range that should contain the true rate for about 9 counties out of 10. An interval that is too narrow misleads the client; one that is too wide is useless to them.

> **The response is synthetic**
>
> The predictors are real county measures from County Health Rankings & Roadmaps (2025 release). The EV rate is **simulated**: I generated it from a secret regression model plus noise. It is *not* real EV data, so do not quote it as a fact about any county. Because a true model exists, you can be scored against the truth, and at the end of the term I will reveal the model.

2 The data

You get three files (on Canvas):

File	What it holds
`train.csv`	2,197 counties: IDs, 20 predictors, and the response `ev_rate_per_1k`.
`test_predictors.csv`	941 counties: IDs and the same 20 predictors, **no response**. These are the counties you predict.
`codebook.csv`	One row per column: name, description, units, and the County Health Rankings measure code.

Things to know before you start:

- `county_id` is a random code (like `C0412`). It carries no geographic meaning.
- `state_id` is an anonymized state (like `S17`). Counties with the same `state_id` are in the same state. There are 51 of them (50 states and DC), and every state in the test file also appears in the training file.
- **Proportions are stored between 0 and 1** (0.25 means 25%), except where the codebook says otherwise.
- The response ranges from well under 1 to a few hundred EVs per 1,000 vehicles, and it is strongly right-skewed. That is one reason it is **scored on the log scale** (Section 4).
- Some predictors matter and some do not. Figuring out which, and in what form, is the project.

3 What you submit

3.1 The file

One CSV file with **exactly one row per test county**: the county's ID, your point prediction, and the two ends of your 90% prediction interval. You may submit on **either scale**, and the column names tell the scorer which one you used:

If your numbers are…	Use these column names
EVs per 1,000 vehicles (the rate itself)	`county_id, pred, lower, upper`
natural logs of the rate	`county_id, log_pred, log_lower, log_upper`

Either way, the scorer works on the log scale (Section 4): it takes the logs of a rate-scale file and uses a log-scale file as it is. So **if you model `log(ev_rate_per_1k)`, submit your model's output directly** in the log format. There is nothing to transform.

Rules for the values:

- `county_id` is copied from `test_predictors.csv`: every test county, once, in any order.
- In every row, the prediction lies inside its interval (`lower` ≤ `pred` ≤ `upper`).

- On the rate scale, all three numbers must be positive (a rate cannot be negative, and the scorer takes logs).
- Extra columns are ignored.

> **Moving an interval between scales**
>
> You will often build an interval on one scale and want it on the other. (The M1 note asks you to think about exactly this.) The rule is short:
>
> **The ends of an interval carry over through any increasing transformation.** If $P(l \le \log Y \le u) = 0.90$, then $P(e^l \le Y \le e^u) = 0.90$, because $\log Y \le u$ happens exactly when $Y \le e^u$. So exponentiate both ends and you have a 90% interval for the rate; take logs of both ends to go back.
>
> Two consequences worth noticing:
>
> - An interval that is symmetric on the log scale is **not** symmetric on the rate scale: the log-scale interval 2 ± 0.5 becomes $e^{1.5}$ to $e^{2.5}$, about 4.5 to 12.2, around $e^2 \approx 7.4$ (2.9 below, 4.8 above).
> - A normal-theory interval built on the rate scale can have a **negative** lower end for a right-skewed response. The log scale never does.
>
> What does *not* carry over cleanly is the mean: $e^{E(\log Y)}$ is the *median* of Y (when $\log Y$ is symmetric), not its mean. For this Cup that does not matter, because the scorer takes the log again, but it matters whenever you report a back-transformed prediction to a client.

A minimal workflow in R (the helper `write_submission()` writes the file and checks it):

```
source("cup_check.R") # on Canvas, next to the data
test <- read.csv("test_predictors.csv")

# ... fit your model on train.csv, then compute pred, lower, upper
#     for every test county, in the order of test$county_id ...

# If your model predicts log(ev_rate_per_1k):
write_submission(
  test$county_id,
  pred,
  lower,
  upper,
  scale = "log",
  file = "team_submission.csv"
)
# If it predicts the rate itself, use scale = "rate" instead.

check_submission("team_submission.csv") # the same checks the app runs
```

3.2 Checking and submitting

The Cup app has two buttons:

- **Check my file** runs every format check and tells you, in plain language, what (if anything) is wrong. It does not score the file and does **not** count toward your limit. Use it as often as you like. `check_submission()` runs the same checks on your own computer.
- **Submit for scoring** checks the file, scores it, shows your team its scores right away, and updates the public leaderboard within about a minute. This **does** count toward your limit.

Your team. Each team gets a private **team code** from me. The first time your team submits, choose "Our team has not submitted yet", enter your code, and pick a **team name** (3–30 characters; keep it classroom-friendly; I may rename anything that is not). After that, pick your name from the list and enter your code.

Limits.

- At most **2 scored submissions per team per week**. A week runs Monday 12:00 am to Sunday 11:59 pm, Pacific time. Unused submissions do not carry over.
- Milestone submissions count toward the limit. **M1 needs two submissions**, so it uses that whole week: check both files first.
- A file that fails the checks is not scored and does not count.
- Submissions open **Monday, October 12** and close **Tuesday, December 1, at 12:00 pm (noon)** Pacific. The app refuses scored submissions after that moment.

Your final entry is your last scored submission before the deadline. Whatever you submit last is what counts for the final standings and the final grade, so make sure your last submission is the one you want.

4 How you are scored

Everything is scored on the **log scale**: the scorer replaces the true rate, your prediction, and your interval endpoints by their natural logs (a log-format file is already there). On the log scale, a miss measures *relative* error: predicting 10 when the truth is 12 costs the same as predicting 100 when the truth is 120. Without the log, a few counties with rates in the hundreds would decide the whole Cup.

Below, y_i, $\hat{y}_i$, l_i, and u_i are the **logged** truth, prediction, and interval ends for county i.

4.1 The benchmark

Every score is relative to a deliberately simple **benchmark**: predict the same number for every county (the training mean of the log rate) with the same normal-theory 90% interval for every county (built from the training standard deviation of the log rate). You will build this benchmark yourselves in Milestone 1. A score of **0 means "no better than the benchmark"**, and higher is better.

4.2 Point skill

For the predictions, add up the squared misses and compare with the benchmark's:

$$\text{point skill} \;=\; 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \hat{y}_0)^2},$$

where $\hat{y}_0$ is the benchmark prediction. This is an **out-of-sample** R^2. A point skill of 0.5 means your squared misses are half the benchmark's. It can be negative if you do worse than the benchmark. The best you could ever expect is the skill of the true mean, which is below 1 because the noise is real.

4.3 Interval skill

Each interval gets an **interval score** (lower is better). With $\alpha = 0.10$ for a 90% interval,

$$\mathrm{IS}_i \;=\; \underbrace{(u_i - l_i)}_{\text{width}} \;+\; \underbrace{\tfrac{2}{\alpha}\,(l_i - y_i)\,\mathbf{1}\{y_i < l_i\}}_{\text{miss below}} \;+\; \underbrace{\tfrac{2}{\alpha}\,(y_i - u_i)\,\mathbf{1}\{y_i > u_i\}}_{\text{miss above}}.$$

In words: **you pay for the width of your interval, and if the truth lands outside, you also pay 20 times the distance by which you missed.** Then

$$\text{interval skill} \;=\; 1 - \frac{\sum_i \mathrm{IS}_i}{\sum_i \mathrm{IS}_{0,i}},$$

where $\mathrm{IS}_{0,i}$ is the benchmark interval's score.

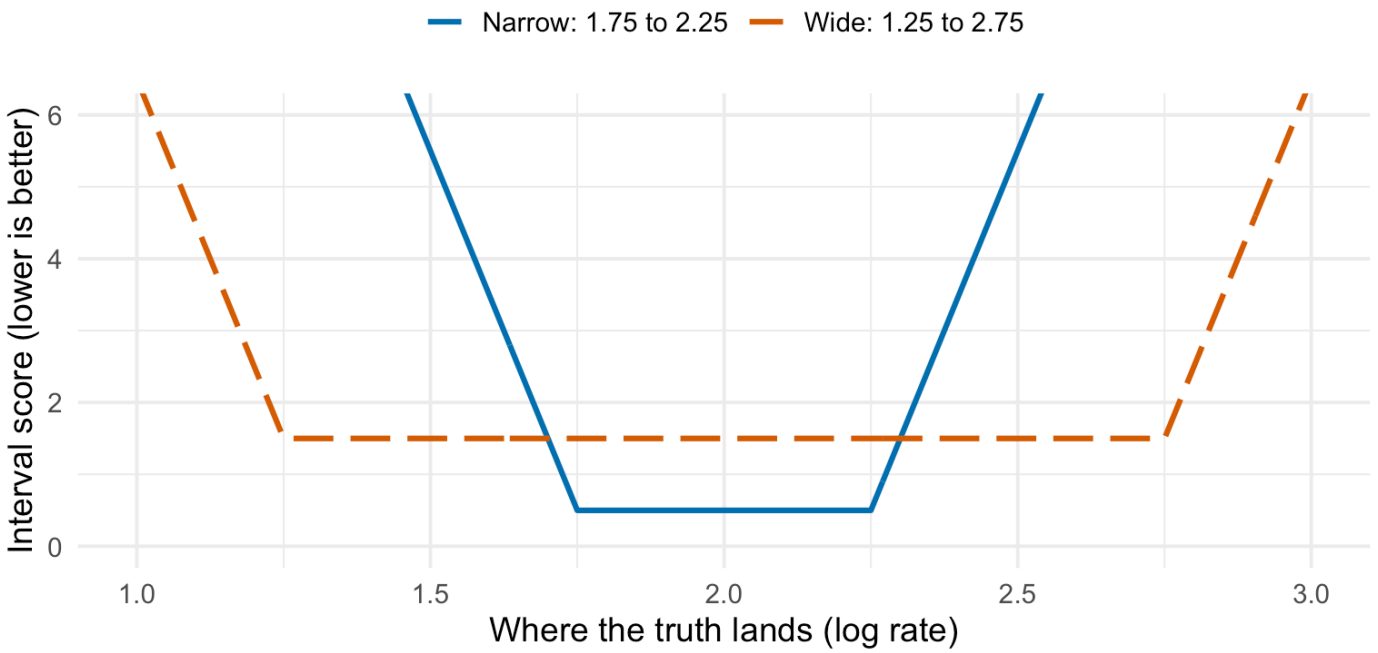


Figure 1: The interval score for one county, as a function of where the truth lands (log scale). Inside the interval you pay only the width. Outside, the cost climbs steeply: 20 times the distance missed. The narrow interval is cheaper when it catches the truth and far more expensive when it misses.

Why this score? It rewards **honest** intervals. In expectation, the best possible interval is the true 5th-to-95th percentile range for that county: shrinking it makes misses too costly, and widening it wastes width. So the way to win is not to game the width, it is to model the uncertainty well, including the fact that it may be larger for some counties than for others.

4.4 A small worked example

Suppose there were only three test counties, and the benchmark predicted 6 for each, with interval 2 to 20. A team submits:

Table 1: A three-county example. The scorer logs everything; the last four columns are computed on the log scale.

County	Truth	Prediction (interval)	Squared miss	Benchmark squared miss	Interval score	Benchmark interval score
A	12	10 (6 to 17)	0.033	0.480	1.04	2.30
B	3	4 (2.5 to 7)	0.083	0.480	1.03	2.30
C	40	15 (8 to 28)	0.962	3.599	8.39	16.17

- County A: the truth (12) is inside 6 to 17, so the team pays only the width, $\log 17 - \log 6 = 1.04$.
- County C: the truth (40) is above 28, so the team pays the width, $\log 28 - \log 8 = 1.25$, plus $20 \times (\log 40 - \log 28) = 7.13$, for a total of 8.39. One bad miss costs more than the other two counties combined.

Totals: point skill $= 1 - 1.078/4.560 = 0.76$, and interval skill $= 1 - 10.46/20.77 = 0.50$. This team beat the benchmark on both.

4.5 The Cup score and the leaderboard

- **Cup score** = the average of point skill and interval skill. Ties are broken by interval skill.
- **Public and private halves.** The test counties were split at random into two halves of about 470 each; you do not know which county is in which. The leaderboard shows scores on the **public half** only. The **final standings use the private half**, revealed at the Regression Bowl (Wednesday, December 2). This keeps the leaderboard honest: a model tuned to the public half's quirks will slide on the private half. (You will see exactly why in week 8.)
- **The ± numbers** are bootstrap standard errors: how much a score would wobble from county-to-county luck alone. Two teams whose scores differ by less than about twice that are, statistically, tied.
- **Coverage** is the share of public counties whose true rate landed inside your interval. For honest 90% intervals it should be near 0.90.
- The leaderboard shows **each team's most recent submission** (your current entry), plus a history of every submission. Two reference rows are always there: the **benchmark** (scores 0 by construction) and an **instructor reference** model built with tools from this course. Beating the reference is possible but not easy.

5 Rules

Teams. Teams of 3 (or as assigned). Everyone contributes; peer evaluations are part of the grade.

Models. Only models that are **linear in the parameters**: transformations of the response or predictors, polynomials, splines, interactions, categorical predictors (including `state_id`), weighted or generalized least squares, ridge, and the lasso. **Prediction intervals** may come from any method in the course: normal theory, the bootstrap, or conformal prediction. **Not allowed:** trees, random forests, boosting, neural networks, or other machine-learning models. If you are not sure whether something is allowed, ask before you submit it.

Data. Use only the files provided. Do not bring in outside data and do not try to identify the real counties or states: the response is synthetic, so it would not help, and it is against the rules. You may look at the test *predictors* as much as you like (for example, to see where you would be extrapolating).

Collaboration. Talk about ideas and methods with anyone. Do not share code, files, or predictions across teams.

AI tools. Allowed, as elsewhere in the course, with an **AI-use log** in your final note: what you asked, what you used, and at least one place where an AI tool was wrong.

One account per team. Submitting under another team's name, or trying to get around the weekly limit, is an academic-integrity violation.

6 Milestones and grading

Milestone	Due	What you hand in	Weight
M1	Sun 10/18	Two scored submissions of the constant-mean benchmark with two different 90% intervals, plus a short note (at most half a page): which interval scored better, and why?	3%
M2	Sun 11/8	Your best **one-predictor** model (submitted), plus a one-page memo: choice of predictor, transformations, unusual points, how you built the intervals, and training R^2 vs. test skill (the gap is optimism).	4%
M3	Sun 11/15	A multiple regression model (submitted), plus a diagnostics memo (at most 2 pages) that includes a fake-data check of your intervals' coverage. For each predictor you keep, show that its effect survives adding `state_id`: how did its coefficient and t-statistic change?	5%
Final entry	**Tue 12/1, 12:00 pm**	Your last scored submission before this moment. **Not late-able.**	5%
Final note	Fri 12/4	About 2 pages: final model, key decisions with evidence (including cross-validation evidence that each predictor you kept improves prediction), and the AI-use log.	5%
Peer evaluation	Fri 12/4	A short Canvas survey on how your team worked.	3%
		Total	**25%**

Late work. Memos and notes are due at 11:59 pm on the listed day and are submitted on Canvas, one per team. In each memo, give the submission number(s) the app showed you, so we score the same files you describe. The requirements are the same for STAT 464 and STAT 564.

Each piece is scored out of 10 points and then scaled to its weight. The full rubric, including the memos, the final note, and the peer evaluation, is on Canvas > Modules > The Regression Cup. The project is 25% of the course

grade.

7 Frequently asked questions

Can we see our private-half score? No. It is revealed at the Regression Bowl.

We submitted something by mistake. Can you delete it? No: it counts. That is what the free **Check my file** button is for. But your final entry is your *last* submission, so a bad submission early on costs you nothing at the end.

Which scale should we submit on? Whichever your model works on; the scores are identical. If you model the log rate, use the log format and skip the transformation. If you model the rate itself, think about how you are scored (squared error on the log scale): which feature of the distribution of $\log(\text{rate})$ minimizes expected squared error, and does your rate-scale prediction deliver it?

The app says our values "look like logs" (or "look large for logs"). What do we do? Check the column names. Rate-scale numbers go in `pred, lower, upper`; log-scale numbers go in `log_pred, log_lower, log_upper`. A mix-up scores terribly, so the app asks before it counts the submission.

Our interval is not symmetric around our prediction. Is that a problem? No. On the rate scale it usually should not be (see "Moving an interval between scales").

Can we use `state_id`? Yes. It is a predictor like any other (a categorical one).

Can we use the test predictors to choose transformations or standardize variables? Yes. You never see test responses, so this is not cheating.

Does the leaderboard rank affect our grade? No. Thresholds decide the grade. The rank decides who lifts the trophy.

What if the app is down near a deadline? Email me your file before the deadline; the timestamp on the email counts. Do not wait until the last hour.